SPLITZY

Technical Specification + User Flow Architecture

A PWA-first money management platform

01 Shared expense circles

02 Personal money tracking

03 Savings projections

04 PDF reporting

05 Future settlement integrations

Next.js / React 19 / TypeScript / RTK Query / MUI / PostgreSQL / Vercel

Architecture proposal - not a production security certification

01 / Product scope

A lean MVP with explicit future growth boundaries.

Product pillars

Split Circles: friend groups, roommates, travel, family and corporate teams; flexible expense allocation and balance views.

Personal Tracker: income, spending, categories, budgets and transfers.

Savings Studio: adjustable monthly or one-time contributions with hypothetical growth scenarios in USD/INR.

Reports: downloadable financial statements and share/export workflows.

Settlements: external/manual payment records initially; real payment processing later.

Non-goals for MVP

No in-app transfers, bank account aggregation, investment execution, regulated recommendations, custom payment gateway, or automatic AI transaction execution.

Global conventions

Single account can join multiple circles. Every shared expense belongs to one circle, one payer and a list of participants. Amounts include currency, exact allocation and immutable audit metadata. Private transactions must never become visible to circle members by accident.

MVP SUCCESS CRITERIA

Users can create a circle, split an expense accurately, view net balances, record manual settlement, track personal budgets, model savings and export a statement.

02 / Technical stack

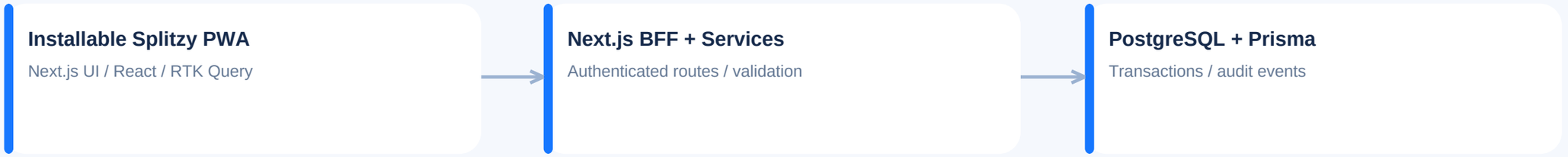
Open-source application code with a managed Vercel deployment.

Layer	MVP technology	Scope / rationale
Web + PWA	Next.js App Router / React 19	Route handling, rendering, installable web experience
Language	TypeScript (strict)	Typed domain contracts and UI components
State	Redux Toolkit + RTK Query	Client preferences and server API caching
UI + style	MUI Core + custom tokens	Accessible data-rich components; premium blue theme
Motion	Motion + CSS transitions	Subtle feedback; reduced-motion support
Forms	React Hook Form + Zod	Input parsing, errors and schema validation
Server	Next.js Route Handlers	BFF and modular business services in MVP
Storage	PostgreSQL + Prisma	Transactional consistency and schema migrations
Hosting	Vercel + managed Postgres	Fast CI previews and production deployments
Testing	Vitest / RTL / Playwright	Unit, component, API and E2E verification
Delivery	Docker Compose / GitHub Actions	Local DB, repeatable testing, CI checks

Defer: NestJS split deployment, Redis, BullMQ, Kubernetes, real payment provider and LLM inference until needed.

03 / Runtime architecture + PWA

The server and database remain authoritative for financial state.



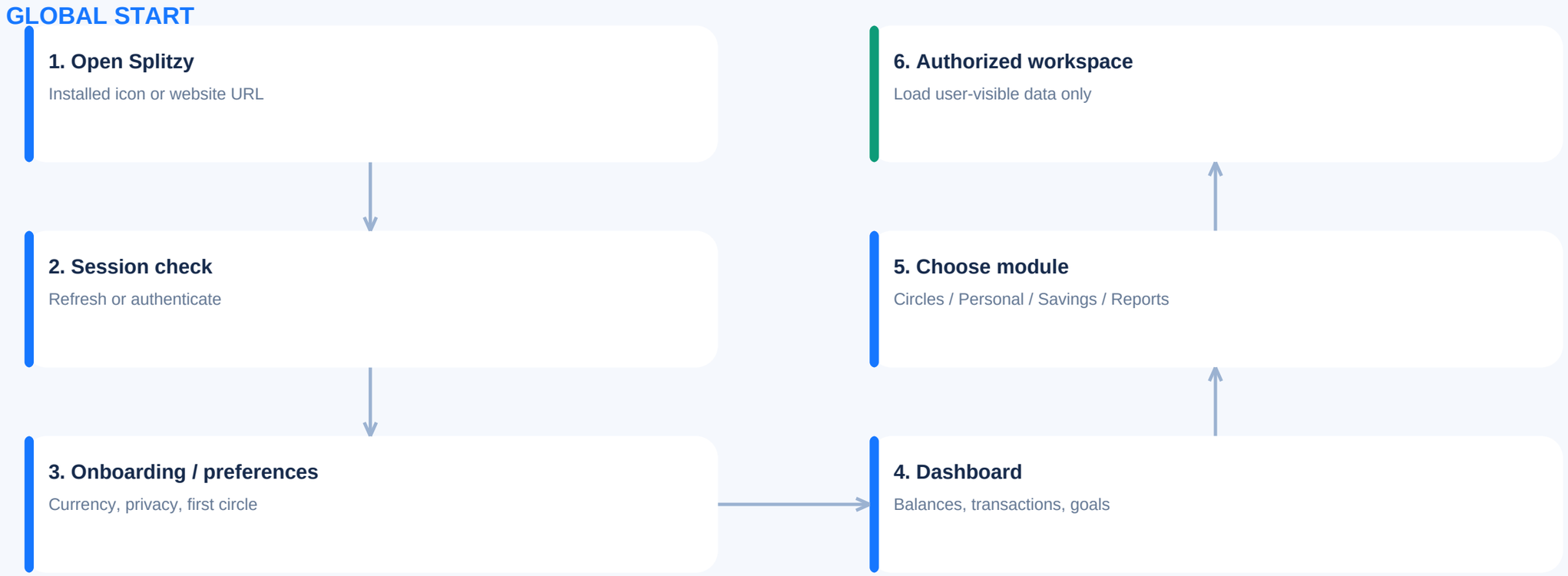
PWA CAPABILITIES

Install	manifest.ts, 192/512/maskable icons, standalone display
Offline	Cache only shell/static assets; treat local entry as pending draft
Updates	Version prompt, service worker lifecycle and safe refresh
Notifications	Consent-based reminders later; feature-detect API support
Security	No generic SW caching of private financial API responses

Data crossing the BFF boundary is authenticated, authorized and revalidated on the server.

04 / Entry and navigation journey

The shared starting point before users enter any feature.



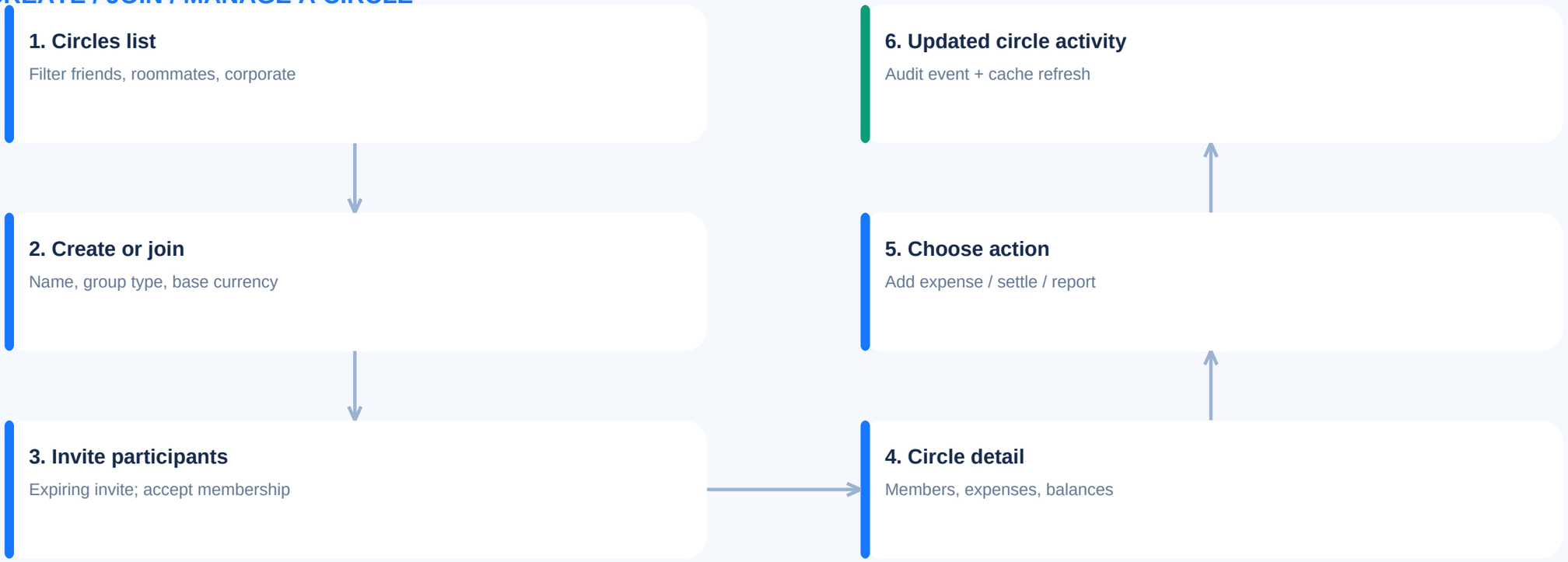
DECISIONS / EXCEPTIONS

If signed out -> login / account recovery. If new -> onboarding. If offline -> shell + clearly pending drafts. If access denied -> safe error with no leaked data.

05 / Circles and group membership

Entry: Dashboard > Circles > Create or select circle.

CREATE / JOIN / MANAGE A CIRCLE



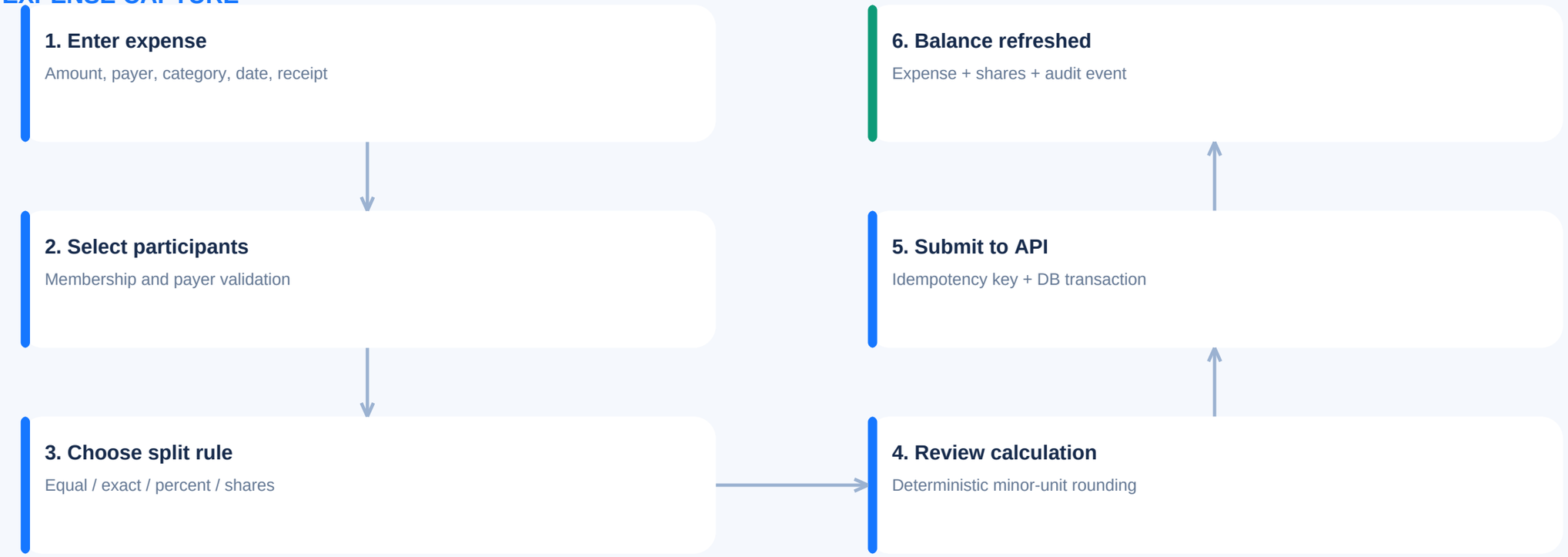
DECISIONS / EXCEPTIONS

Invitation invalid -> display retry; member removed -> revoke access; corporate workspace requires additional role and tenant authorization.

06 / Add expense and split calculation

Entry: Circle detail > Add expense.

EXPENSE CAPTURE



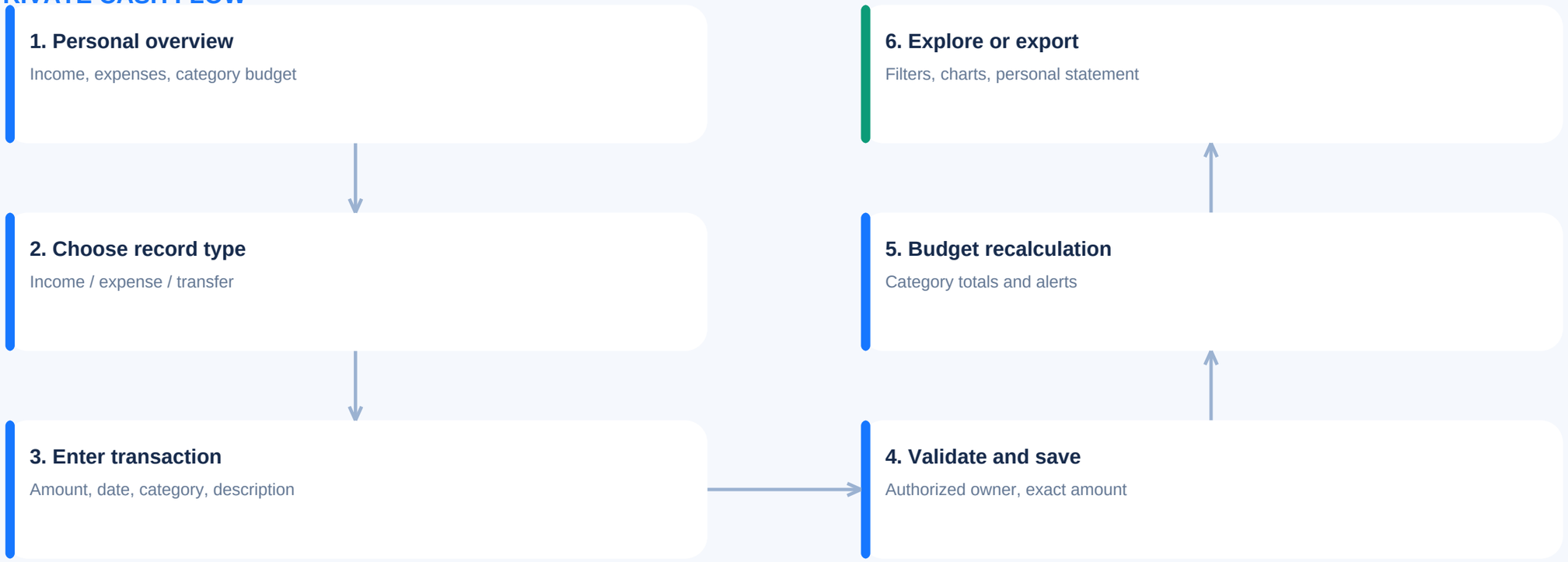
DECISIONS / EXCEPTIONS

Invalid total -> block save. Duplicate request -> return original result. Offline entry -> draft only. Reversal/edit -> history preserved, recompute authoritative balances.

07 / Personal tracker journey

Entry: Dashboard > Personal > Add transaction or view budget.

PRIVATE CASH FLOW



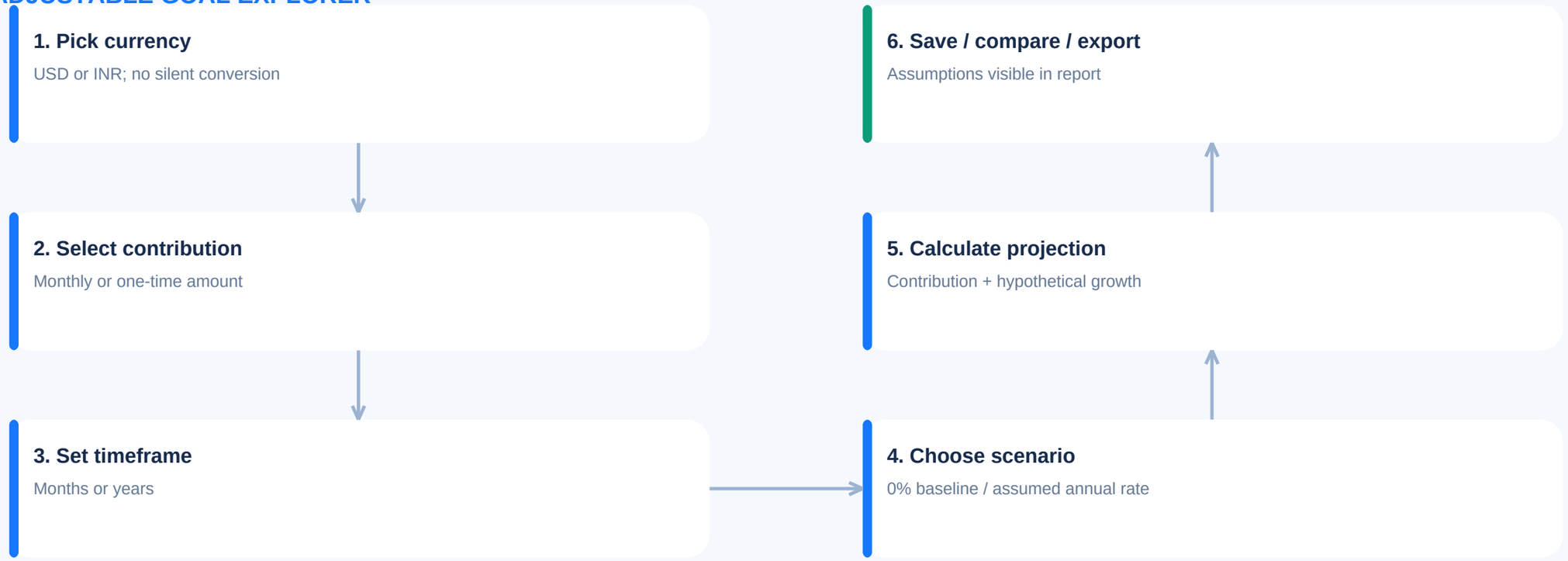
DECISIONS / EXCEPTIONS

Transfers and reimbursements are not automatically classified as new income. Personal records remain private from circle and corporate members.

08 / Savings Studio journey

Entry: Dashboard > Savings > New scenario.

ADJUSTABLE GOAL EXPLORER



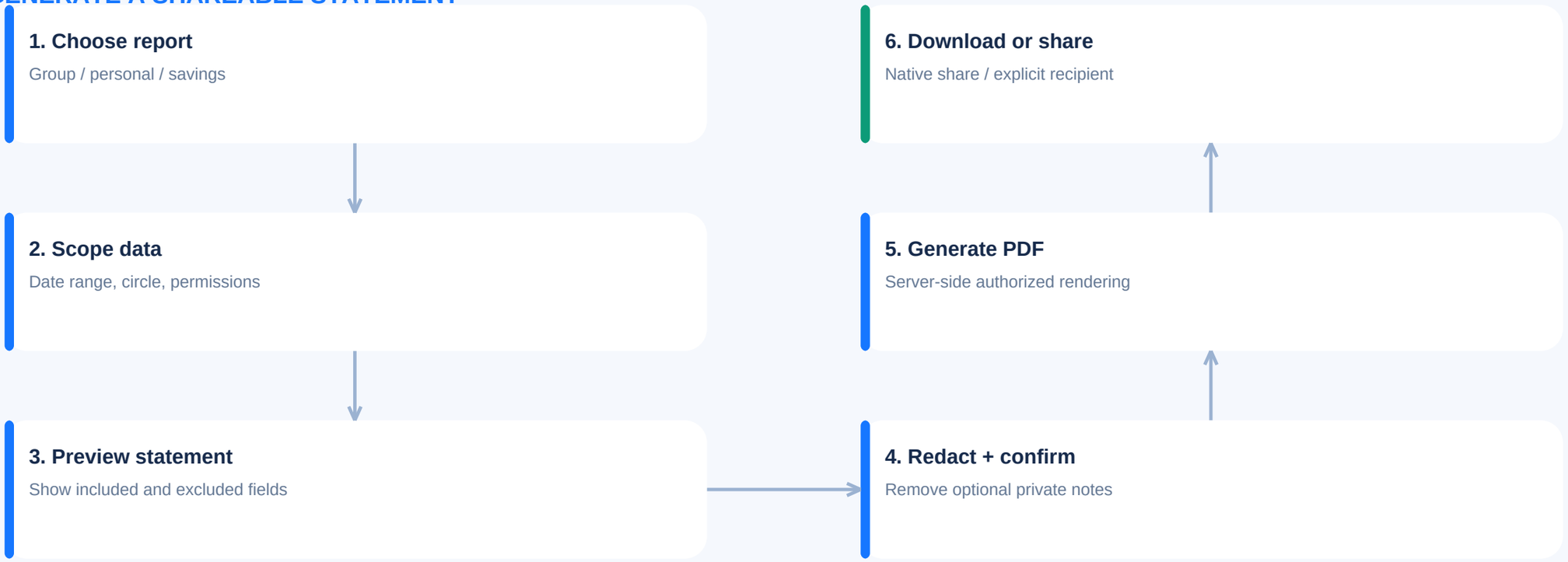
DECISIONS / EXCEPTIONS

Zero-return case is cash contributions. Growth is illustrative, not guaranteed. No trades, deposits or personalized investment recommendations in MVP.

09 / Report and sharing journey

Entry: any module > Export; or Dashboard > Reports.

GENERATE A SHAREABLE STATEMENT



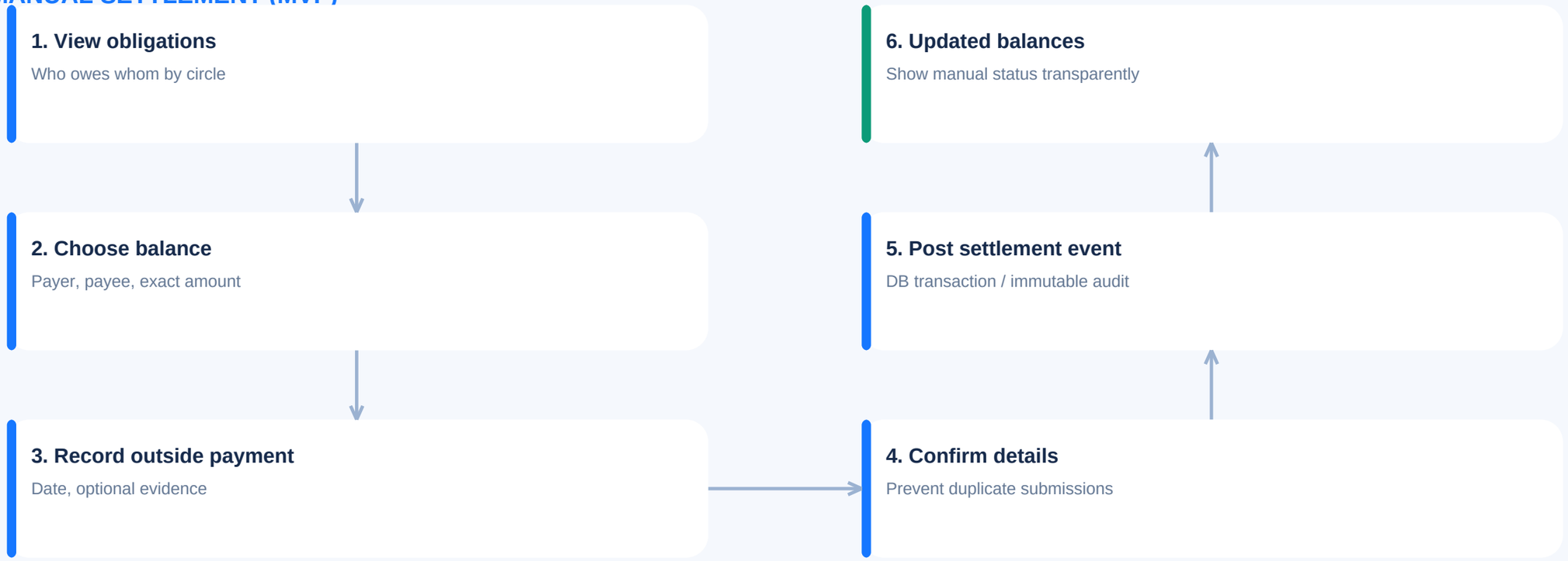
DECISIONS / EXCEPTIONS

Never expose private report URLs publicly by default. Future hosted links require short expiry, authorization and revocation. Reports include timestamps and disclaimers.

10 / Settlement journey

Entry: Circle > Balances > Settle Up.

MANUAL SETTLEMENT (MVP)



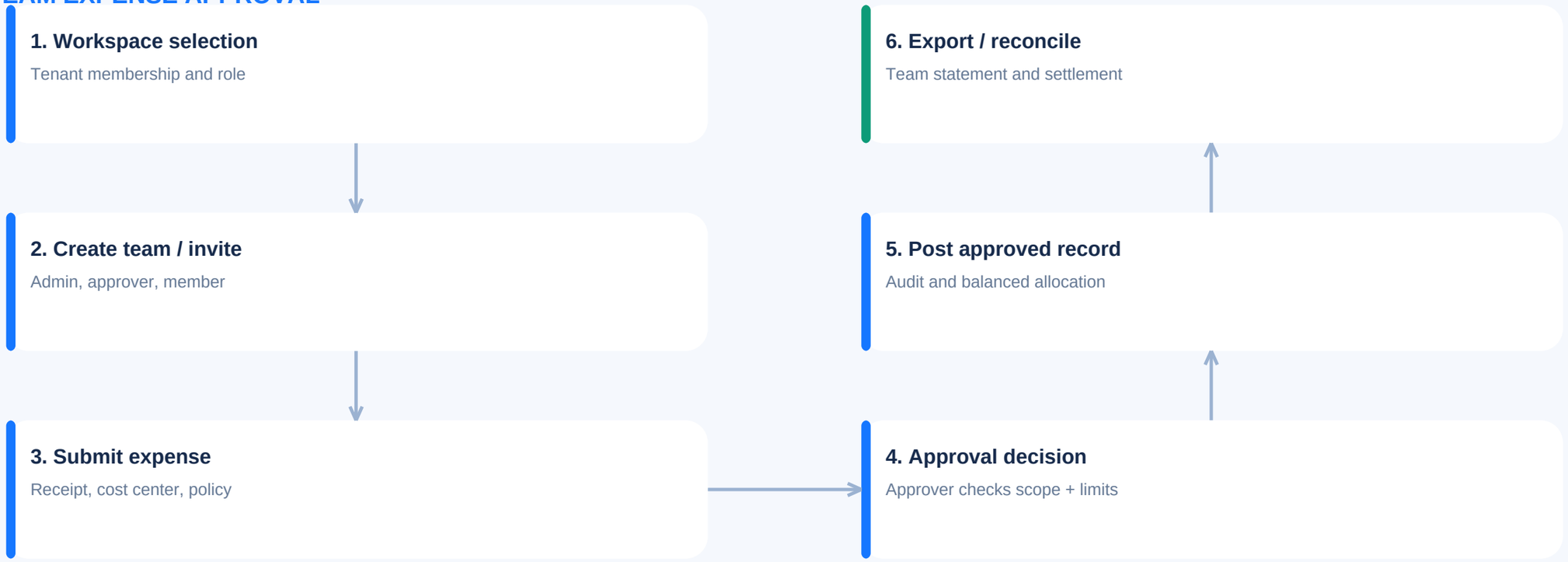
DECISIONS / EXCEPTIONS

Never claim that Splitzy moved funds. Future provider flow: create intent -> authorize -> verify webhook -> reconcile -> post final ledger state. Failure stays pending.

11 / Corporate workspace journey

Entry: Dashboard > Workspaces > Corporate team.

TEAM EXPENSE APPROVAL



DECISIONS / EXCEPTIONS

Rejected expense -> return to submitter. Admins cannot read private personal transactions. Approval thresholds and separation of duties are configurable.

12 / Security and correctness

Financial credibility depends on authorization, precision and traceability.

Identity & sessions

Managed OIDC, MFA/passkeys where supported; HttpOnly + Secure + SameSite cookies; rotation, logout and step-up verification.

Access control

Verify ownership, circle membership and workspace role on every server read and mutation, including reports and file downloads.

CSRF / XSS / CSP

CSRF defense for cookie-authenticated writes, safe encoding, restrictive CSP, secure headers and safe upload validation.

Money ledger

Exact integer minor units or decimal arithmetic; deterministic remainder rules; ACID writes, idempotency and reversing entries.

Data protection

TLS, private object storage, secrets manager, encryption at rest, audit logs, backups and tested restore procedures.

PWA boundaries

Sensitive financial responses are not broadly cached; offline creates drafts only; online server always confirms final state.

Validation gates

OWASP ASVS-inspired controls, SAST/DAST, dependency and secret scans, permissions regression tests and pen testing before payments.

13 / Delivery plan and acceptance gates

Build locally, release to Vercel and keep the production path reversible.

1

Phase 1 - Foundation

Next.js PWA scaffold, design system, auth, Postgres, Prisma, CI/CD, environment isolation.

2

Phase 2 - Core finance

Circles, exact expense allocation, balances, manual settlement, audit trail.

3

Phase 3 - Personal + savings

Transactions, budgets, projections, currency formatting and transparent assumptions.

4

Phase 4 - Reports + beta

PDF generation, privacy review, responsive UX, PWA installation and offline handling.

5

Phase 5 - Future scale

NestJS extraction if justified, background workers, corporate approvals, provider payment settlement.

CI PIPELINE

PR > Typecheck / unit > API & E2E > dependency scan > Vercel preview > approval > production + rollback

Release criteria: correct sums, role isolation, offline clarity, secure exports, tested restore and no payment false positives.